Research Progress Report: Requirements, Data Profile, and Design of PROBE-CASCADE Through M3

Multi-LLM Comparison Platform: From Model Comparison to a Cost-Aware Router That Generalises to New Models

Student: Venu Dattathreya Vemuru (811776500)
Course: CSCI 7200, Master's Project
Report period: 1 October to 10 November 2026 (since Research Progress Report 7)
GitHub: https://github.com/venu284/LLM_Comparison.git

---

1. Executive Summary

Report 7 ended with a design, PROBE-CASCADE, and a plan to build it in eight milestones (M0 to M8). This report covers the setup milestone M0, a profile of the main evaluation corpus, and a requirements analysis and repository design for the next three milestones: the data layer (M1), the replay harness and baselines (M2), and the IRT core with new-model onboarding (M3).

There are no router results in this report. The implementation of M1 to M3 starts after it. Every number below is either a fact about the repository, a descriptive statistic of the public CodeRouterBench data, or a check against numbers the ACRouter authors published. Section 8 describes the analyses M1 to M3 will produce, as empty tables with the decision each one feeds.

What happened in the period:

1. **Setup (M0, 5 October).** Docker runs, the `router/` package skeleton exists, and 84 tests pass. Every reference identifier in Report 7 resolves, and seven titles need correcting.
2. **Novelty check (M0, 5 October).** Two papers, UniRoute and ICL-Router, already publish curves of routing quality against the size of the onboarding set. We retract Report 7's claim to be the first such measurement and narrow the contribution (§5).
3. **Two suspected defective tasks are not defective.** The reference solutions for API-013 and CSS-012 pass 12/12 and 8/8 tests. Report 7's "defective task" bucket is therefore unsupported.
4. **CodeRouterBench profile (10 November).** Profiling the released data changed four requirements before any code was written (§3):
    - the in-distribution tasks ship without prompt text, which the difficulty model needs;
    - 41% of scores are continuous, which the binary IRT model does not handle as designed;
    - the paper's tables were computed on an older matrix with 881 missing cells;
    - the ACRouter GitHub repository no longer resolves, so we cannot use its scripts.
5. **Reproduction, first pass.** From the released files alone we reproduce three published numbers exactly: Always-Opus (43.83 AvgPerf, 1.29 Perf/$), the released ACRouter decisions (50.14, $22.31), and the paper's oracle (57.00, on the older matrix). One number does not reproduce: DimensionBest. It stays open for M2.

The schedule has slipped. The plan had M1 to M3 finished by 2 November. Only M0 is done, and no implementation work happened between 5 October and 10 November. Section 10 gives a revised timeline and says what gets cut.

---

2. Work Completed

2.1 Repository cleanup (5 October)

Before implementation we cleaned up the repository so new code would sit on a stable base:

- merged the three open branches (data sufficiency, goal lock, Report 7) into `main`;
- froze the 325 Phase 5 runs as `data/phase5_results.csv`, with its SHA-256 in `data/README.md`;
- made evaluation runs write to the database only when given `--db`, so a test run cannot touch Phase 5 rows;
- added a root `pyproject.toml` with the test configuration. 73 tests passed at that point.

While re-running the analysis we found and fixed a bug in the failure taxonomy. Extraction failures were labelled only when the recorded test total was zero. After the fix, the `drop_artifactual` option drops 21 rows, not 20. The RQ1 result moves from p = 2.6e-4, Cramér's V = 0.265 to p = 2.4e-4, V = 0.267. The conclusion does not change. The Phase 6 headline numbers reproduce: oracle 58.5%, best single model 53.8%, recommender 36.9%.

2.2 M0: setup and novelty check (5 October)

Item	Outcome
M0.1 Docker	Docker 29.2.0 runs. The image was rebuilt and both suspected tasks were run against their reference solutions (§2.3)
M0.2 Skeleton	`router/` with eight subpackages and `settings.py`; `tests/router/test_skeleton.py` checks the import rules. 84 tests pass
M0.3 Merge	Done in the cleanup above
M0.4 Novelty	Gate tripped in part. Contribution narrowed (§5)
M0.5 References	All identifiers resolve on arXiv. Seven titles are wrong (§4)

The M0 work is on the branch `worktree-m0-setup` and is merged into this report's branch.

2.3 API-013 and CSS-012 are not defective tasks

Report 7 put 10 of the 63 zero-test rows in a "defective task" bucket, because every model scored zero tests on API-013 and CSS-012. With Docker working we ran the reference solutions through the same harness. API-013 passes 12 of 12 jest tests and CSS-012 passes 8 of 8 Playwright tests. The tests were last changed on 5 April 2026, before Phase 5 ran on 27 April, so the models were scored on these same tests.

All 10 Phase 5 runs on these two tasks have `extraction_success = True` and `tests_total = 0`. The test suite collected no tests on the models' code. Something in how model output was placed or named broke test collection, and we have not found out what. Until we do:

- the "defective task" label for these 10 rows is unsupported;
- whether `drop_artifactual` should still drop them is open, and the RQ1 numbers under that option may move again;
- the `keep` analysis (p = 1.9e-5) does not depend on this question.

This is now a tracked item in the plan, due before the live layer (M6) reuses the test runner.

2.4 Novelty check

Report 7 scheduled a final prior-art check before claiming the onboarding study as new. We read FlyRoute, "Routing with Generated Data", RouteProfile, and the abstracts of all 11 papers that cite ACRouter. We read the cited sections of UniRoute and ICL-Router in full text. MonoRouter could not be found on arXiv or the web.

- **UniRoute** (2502.08773, Fig. 2) plots deferral quality for unseen models against the number of validation prompts used to represent them. It is cost-aware and includes LiveCodeBench. Its K-NN grid starts at k = 5, so small k is likely covered.
- **ICL-Router** (2510.09719, §4.6) reports routing accuracy for 100, 300, 500 and 1,000 profile exemplars per model: 74.94% at 100, peaking at 76.30% at 500. It covers MBPP and HumanEval, with no cost, no cascade and no online updates.
- **IRT-Router** (2506.01048, §6.3) onboards a new model (Claude 3.5 Haiku) from a text profile alone, at accuracy 0.67, and lists few-shot cold start for new models as future work.
- **RouteProfile** (2605.00180) builds profiles from metadata without probing. It becomes our text-only comparator.
- **FlyRoute** (2605.22057) uses an LLM router over agent profiles and proprietary data.
  RGD/CASCAL (2601.09692) estimates skills from generated data and reports no k curve. None of ACRouter's 11 citers measures a k curve. ORACLE (2607.22465) routes with verifier feedback and is related work for the verified cascade.

The verdict and the narrowed claim are in §5.

2.5 CodeRouterBench profile (10 November)

We downloaded the public CodeRouterBench release (Hugging Face `Lance1573/CodeRouterBench`, last modified 23 June 2026) and profiled it with four short scripts. These scripts and their captured output are in `router/experiments/m1_profile/`. The next section reports what they found. No model was called.

---

3. What the Data Profile Found

3.1 The corpus

Split	Tasks	Models	Rows	Use in our plan
Probing (train + validation)	7,080	8	56,640	IRT fit, anchor pool, difficulty training
ID test	2,919	8	23,352	Baseline reproduction, held-out evaluation
OOD176	176	8	1,408	Out-of-distribution check (bug fixing 162, code generation 14)

The in-distribution (ID) tasks fall in **9 dimensions**, each with 764 to 805 probing tasks: algorithm, bug fixing, code completion, code generation, code refactoring, code understanding, data science, multi-language, and test generation. They come from 11 public source families: DS-1000, CodeXGLUE, Bugs2Fix, DebugBench, CRUXEval, BigCodeBench, LiveCodeBench, MBPP, HumanEval, HumanEval-X (five languages) and CanItEdit.

The eight models and their behaviour on the ID test split:

Model	Price $/M tokens (in / out)	Mean score %	Mean cost $/task	OOD176 resolved %
Claude Opus 4.6	5.00 / 25.00	43.83	0.01165	63.6
GPT-5.4	2.50 / 15.00	42.21	0.00568	64.2
Claude Sonnet 4.6	3.00 / 15.00	41.31	0.00858	43.2
Qwen3-Max	0.34 / 1.38	39.67	0.00052	33.0
GLM-5	0.80 / 3.00	38.03	0.00571	44.9
Qwen3.5-Plus	0.11 / 0.66	37.16	0.00171	27.3
Kimi K2.5	0.01 / 2.90	36.66	0.00080	19.9
MiniMax M2.7	1.00 / 3.00	35.62	0.00513	35.2

Prices are from the release's `models.json`. Five of them were converted from CNY by the authors, which Rule R7 requires us to record and stress-test.

Two things in this table matter for the router:

- **On ID tasks the models are close.** All eight are within 8.2 points of each other. Qwen3-Max is 4.2 points behind Opus at 1/22 of the cost per task. That is the situation a sufficiency router is built for.
- **On OOD tasks they are not.** The resolve rates range from 19.9% to 64.2%, and the cheap models fall furthest. A router that only learned the ID picture would under-escalate on agentic bug fixing.

3.2 Headroom and nesting

The numbers in this subsection use ground-truth scores. They are clairvoyant upper bounds, not anything a router can reach (Rule R1).

- **Oracle.** Choosing the best model per task gives 60.02 AvgPerf on the complete ID test matrix, 16.2 points above Always-Opus. In 24.3% of tasks no model scores above zero.
- **Nesting.** We counted a task as solved at score ≥ 0.5. Opus solves 1,270 of 2,919 tasks, and some model solves 1,771. Of each other model's solved tasks, 78% to 90% are also solved by Opus. This pool is less nested than our Phase 5 pool, where GPT-OSS-120B solved 35 of the 38 tasks any model solved (92%). So selection routing has more room here than it had on our data.
- **Cost.** Picking the cheapest model that reaches each task's best score costs $0.00205 per task, against $0.01165 for Always-Opus, which is 82% less. Report 7 showed that most of a clairvoyant saving like this disappears once the router has to predict instead of look. E2 measures the realistic share.

3.3 Requirement changes found by profiling

R-a. ID tasks have no prompt text. The ID task files carry only `task_id`, `dimension` and the split. The raw matrices have no prompts either. Only the OOD176 tasks include prompts. Our difficulty model (Layer 2, M3.2) predicts a task's difficulty from an embedding of its prompt, so M1 must rebuild the prompts by joining each `task_id` back to its public source dataset. The ids encode the source and index, for example `dim1_mbpp_631` or `dim1_bcb_BigCodeBench/767`. Eleven source families means eleven join adapters, and some will not join cleanly. The "_sup" subsets and the LiveCodeBench contest ids are the likely problems. The M1 gate now includes a join rate.

R-b. 41% of scores are not pass/fail. Only code generation and data science are strictly 0/1. Algorithm, code completion and test generation are mostly 0/1 with a few partial levels. Bug fixing, code refactoring, code understanding and multi-language are mostly continuous. The 1PL and 2PL IRT models in the design assume a binary outcome, $P(\text{pass}) = \sigma(\theta - b)$. M3 therefore needs a stated rule. The candidates are:

- binarise at a fixed threshold and report sensitivity to it;
- fit IRT on the five mostly-binary dimensions only and keep the other four for the cost study;
- use a continuous-response IRT variant.

We will choose by held-out log-likelihood on the probing split, and the choice also fixes what "accepted" means for CPAT.

A continuous score does not mean an LLM judge. CodeXGLUE and Bugs2Fix are normally scored with text-similarity metrics. Report 7 said three dimensions were judged by an LLM. We have not confirmed which ones, and Rule R4 (primary evidence from execution-scored data only) depends on it. M1.2 will settle the mapping from the source benchmarks' own scoring rules.

R-c. The paper's tables use a different matrix than the release. The released CSV files are complete: every task has a score for all eight models. The paper's Table 3 was computed on the raw file

`raw_matrices/phase1_acrouter_v2/obs_matrix.json`. On the ID test split, that file has 22,471 of 23,352 cells. 502 tasks are missing at least one model, and GLM-5 is missing 404 of them. The difference explains the oracle: 57.00 on the ragged matrix, the paper's number exactly, and 60.02 on the complete one. The cells present in both files are identical. M2 must therefore report every baseline on both matrices: the ragged one to check against the paper, and the complete one for our experiments.

R-d. The ACRouter code repository is gone. `github.com/LanceZPF/agent-as-a-router` returned 404 on 10 November, and the account lists no public repositories. The Hugging Face release still has the data, the decision files and the metric files, but not the scripts. The M2 gate can no longer say "reproduce with their released scripts". It now checks our harness against their released decision and metric files, and re-implements the baselines from the paper's description.

3.4 First reproduction checks

Number	Published	Ours, from released files	Status
Always-Opus AvgPerf / Perf/$ (ID test)	43.83 / 1.29	43.83 / 1.29	Reproduced
ACRouter release, ID (AvgPerf / $Total / Perf/$)	50.14 / 22.31 / 2.25	50.14 / 22.31 / 2.25	Reproduced by scoring their decision file
Oracle, ID test	57.00	57.00 on the ragged matrix; 60.02 on the complete one	Explained (R-c)
DimensionBest, ID test	47.50	51.68 (our rule: best model per dimension on probing, applied to test, complete matrix)	**Open**

Scoring the released decisions tells us how Perf/$ is computed (AvgPerf divided by total dollars). The decision file also says what the released ID router does: every decision is labelled `hierarchical_group_model_map`. It is a lookup from the task's source (1,954 tasks), its first three id tokens (961) or its dimension (4) to a model, choosing Opus for 61.3% of tasks. We have not found where this policy is described in the paper, and it is not the LoRA orchestrator Report 7 summarised.

The DimensionBest gap is open. It is not a finding. Our simple rule scores 4.2 points above the paper's DimensionBest and 1.7 points above ACRouter's 49.98. Before reading anything into that, M2 has to rule out the ordinary explanations. The paper's table may use a different definition (a cost-aware choice, or the ragged matrix), and the baseline table notes that its ID column was "copied unchanged from the provided table" rather than recomputed. The M2 gate covers this number.

---

4. Corrections to Report 7

#	Report 7 said	Correction
C1	`drop_artifactual` drops 20 rows; RQ1 p = 2.6e-4, V = 0.265	21 rows; p = 2.4e-4, V = 0.267 (§2.1)
C2	10 rows are "defective task" (API-013, CSS-012)	Reference solutions pass; the label is unsupported and the cause is open (§2.3)
C3	Novelty item 2: "the first measurement of how many probe tasks a new model needs"	Retracted. UniRoute and ICL-Router publish such curves (§2.4, §5)
C4	CodeRouterBench has "10 coding dimensions"	The ID data has 9 dimensions. OOD176 covers bug fixing and code generation
C5	ACRouter limitation 4: "no escalation"	True of the ID system. The released OOD run uses a fixed-order verify-and-escalate cascade: MiniMax, Kimi, GPT-5.4, GLM-5, then Opus only if two cheap attempts apply cleanly
C6	ACRouter OOD: 62.50 vs Always-Opus 57.14 (n = 176)	The release's own table gives 73.30 vs 63.64 on OOD176, with an oracle of 78.98. Its README reports 66.96 on an older 112-task set. M2 records which is canonical
C7	"Code and data are released"	The data is. The code repository returned 404 on 10 November (§3.3, R-d)
C8	Three dimensions are LLM-judged	Unconfirmed. Four dimensions are mostly continuous, and similarity metrics may explain them. To confirm in M1.2 (§3.3, R-b)
C9	Seven reference titles	Corrected in §12: [3], [14], [15], [16], [17], [19], [25]
C10	Open items: Docker unavailable, `pytest` missing, data-sufficiency branch unmerged	All three resolved (§2.1, §2.2)

5. Revised Contribution

Report 7 offered three contributions. The second, measuring for the first time how many probe tasks a new model needs, does not survive the novelty check. The claim we can defend is narrower:

1. **Anchor selection, not sample size.** New models are onboarded with probe tasks chosen by IRT Fisher information. We compare them with random anchors and with a UniRoute-style cluster representation built from the same k probes. IRT-Router names few-shot cold start as future work. UniRoute and ICL-Router vary how many samples they use, not which ones.
2. **Measured where routing is used.** We measure onboarding as stream regret and as cost per accepted task (CPAT, with an acceptance floor) inside a cascade checked by run-time signals. UniRoute and ICL-Router report static routing accuracy or deferral curves.
3. **Live, unseen endpoints.** Free-tier models that the offline router has never seen are onboarded live on our 65 web-development tasks.

UniRoute and ICL-Router are related work and E1 comparators. C5 also narrows our cascade differentiator. ACRouter's release does escalate. What it lacks is escalation ordered by cost and driven by the router's own predicted pass probability, and a way to admit a new model into the order. The research question stays the same: how much measured evidence does a new model need before the router uses it well, and which evidence is worth paying for?

---

6. Requirements Analysis, M1 to M3

6.1 Scope

M1 to M3 build the offline half of PROBE-CASCADE: loading data, replaying task streams, the baselines, and the IRT model that onboarding depends on. They produce no routing policy of our own yet. The policy (Thompson sampling, the τ threshold, escalation) is M4. The headline experiments E1 to E3 are M5.

6.2 Functional requirements

ID	Requirement	Milestone	Acceptance test
FR1	Load CodeRouterBench probing, ID test and OOD176 into one `OutcomeMatrix` (task, dimension, source, model, score, cost, tokens, latency, scoring type)	M1	Row counts 56,640 / 23,352 / 1,408; no NaN scores; 148 zero-token rows flagged
FR2	Load the ragged paper matrix as a second view of the same tasks	M1	22,471 cells on ID test; identical values where both have a cell
FR3	Rebuild ID prompt text from the 11 source datasets	M1	Join rate per source reported; overall ≥ 95%, or the shortfall documented and the tasks excluded from M3.2
FR4	Tag each dimension as execution-scored, similarity-scored or LLM-judged, with the source of the tag	M1	Every dimension tagged; tag cites the source benchmark's scoring rule
FR5	Load the LLMRouterBench code subsets, dropping zero-cost models from economic metrics	M1	Counts match the published statistics; zero-cost models listed
FR6	Load our 325 Phase 5 runs read-only	M1	325 rows; file hash unchanged after a test run
FR7	One price table for every model, with source URL and access date	M1	A price for every model in every loader; currency conversions noted
FR8	Seeded, deterministic task-stream replay, with leave-one-model-out and model-arrival events	M2	Same seed gives identical decisions; a held-out model is invisible before its arrival
FR9	Metrics: AvgPerf, $Total, Perf/$, cumulative regret (reward s − 0.1·USD), CPAT with floor, cost at matched pass rate, bootstrap 95% CIs	M2	Recomputes the reproduced numbers in §3.4 to two decimals
FR10	A simulated run-time verifier that flips ground truth at rate ε	M2	Measured flip rate within ±1 point of ε over 10,000 draws; replay never reads the raw score to escalate (R1)
FR11	Baselines: always-best, random, fixed cheap-to-strong cascade, kNN, DimensionBest, LinUCB, ACRouter (released decisions)	M2	Each runs on both matrices; see the M2 gate
FR12	IRT: 1PL and 2PL fits on the probing split, with a Laplace posterior for each model's θ	M3	Converges; parameter recovery on synthetic data (correlation ≥ 0.95 between true and fitted b)
FR13	Difficulty predictor: prompt embedding to b_q, with a local model (BGE or MiniLM)	M3	Held-out correlation reported against a dimension-mean baseline
FR14	Anchor selection by Fisher information, and random selection for comparison	M3	k anchors returned, no duplicates, deterministic for a seed
FR15	Onboarding: k anchor outcomes to θ_m with mean and standard deviation	M3	sd shrinks as k grows on synthetic data

6.3 Non-functional requirements

ID	Requirement
NFR1	No paid API calls. Everything in M1 to M3 runs offline on released data (user decision of 4 October)
NFR2	Every experiment script is seeded and writes its output next to itself in `router/experiments/out/` (R8)
NFR3	Downloaded data and third-party code live in `external/`, which git ignores. The repository records the dataset revision
NFR4	Router modules never define a top-level `config`, `models`, `analysis` or `db` module, which the `pipeline/` package already uses
NFR5	Tests run from the repository root with `pipeline/venv/bin/python -m pytest`, and each new module ships with its tests
NFR6	The full M1 to M3 pipeline runs on a laptop CPU. Embedding about 10,000 prompts is the slowest step

6.4 Rules carried from the plan

The rules from Report 7 and the plan apply unchanged. The ones M1 to M3 touch most:

- **R1.** No escalation on ground truth during replay; the simulated verifier (FR10) exists for this.
- **R2.** The simple baselines always appear next to any result.
- **R3.** CPAT always carries an acceptance floor.
- **R4.** Primary evidence comes from execution-scored dimensions; FR4 makes this checkable.
- **R5.** Phase 5 data is read-only; FR6 tests it.
- **R7.** Prices carry a source and date, plus a sensitivity sweep.
- **R8.** Multiple seeds and bootstrap CIs on every headline number.

6.5 Milestone gates, revised

Gate	Original (Report 7 plan)	Revised
M1	All three corpora load and tests pass	Same, plus: prompt join rate reported (FR3), every dimension tagged by scoring type (FR4), both matrix views load (FR2)
M2	Our harness reproduces the paper's kNN, DimensionBest and Always-Opus within ±1.0 AvgPerf point, using their released scripts	On the ragged matrix, our harness reproduces Always-Opus, the oracle and the released ACRouter decisions exactly, and kNN, LinUCB and DimensionBest within ±1.0. If a baseline misses, we document why and the gate passes only if the cause is a stated definition difference
M3	θ from k = 50 anchors ranks the 8 models like their full-data pass rate (Spearman $\rho \geq 0.8$), and the ability error falls as k grows	Same, measured per held-out model (leave-one-model-out), plus a scoring rule chosen for the continuous dimensions (R-b)

With eight models, Spearman's ρ moves in coarse steps. $\rho \geq 0.8$ means the squared rank differences sum to at most 16, which allows up to eight adjacent swaps. So we will also report the mean absolute error of $\hat{\theta}$ itself.

---

7. Repository After M3

7.1 Layout

New files are marked +. Files from M0 are marked =. Directories that stay empty until M4 or later are listed at the end.

```
LLM_Comparison/
├── benchmark/                        = 65 web-dev tasks, tests, Docker harness
├── data/phase5_results.csv           = frozen Phase 5 snapshot (read-only)
├── pipeline/                         = data collection and Phase 6 analysis (reused, not rewritten)
├── external/                         + git-ignored
│   ├── CodeRouterBench/              + HF snapshot, revision recorded in router/data/README.md
│   ├── LLMRouterBench/               + code subsets
│   └── sources/                      + DS-1000, MBPP, HumanEval, ... for the prompt join
├── router/
│   ├── settings.py                   = paths, seeds, default τ and k
│   ├── data/
│   │   ├── README.md                 + dataset revisions and download commands
│   │   ├── schema.py                 + OutcomeMatrix and Task dataclasses
│   │   ├── coderouterbench.py        + loaders for the complete and ragged views
│   │   ├── prompts.py                + task_id to prompt join, one adapter per source family
│   │   ├── scoring.py                + per-dimension scoring type and acceptance rule
│   │   ├── llmrouterbench.py         + code subsets, priced models only
│   │   ├── phase5.py                 + read-only wrapper over pipeline/analysis/data.py
│   │   └── prices.py                 + single price table, source and date per entry
│   ├── sim/
│   │   ├── stream.py                 + seeded replay, leave-one-model-out, arrival events
│   │   ├── verifier.py               + simulated run-time verifier with error rate ε
│   │   └── metrics.py                + AvgPerf, Perf/$, regret, CPAT@floor, matched-cost, bootstrap
│   ├── baselines/
│   │   ├── static.py                 + always-best, random, DimensionBest
│   │   ├── cascade.py                + fixed cheap-to-strong cascade
│   │   ├── knn.py                    + kNN over prompt embeddings
│   │   ├── linucb.py                 + cost-aware LinUCB
│   │   └── acrouter_release.py       + replays the released ACRouter decision files
│   ├── core/
│   │   ├── irt.py                    + 1PL/2PL fit, Laplace posterior, Fisher information
│   │   ├── difficulty.py             + embedding to b_q regressor
│   │   ├── anchors.py                + Fisher and random anchor selection
│   │   └── onboarding.py             + k outcomes to θ_m (mean, sd)
│   └── experiments/
│       ├── m1_profile/               = (this report) profiling scripts and outputs
│       ├── m2_reproduce.py           + baseline reproduction table, both matrices
│       ├── m3_irt_fit.py             + 1PL vs 2PL, scoring-rule choice
│       ├── m3_difficulty.py          + difficulty predictor evaluation
│       ├── m3_onboarding_curve.py    + ability error vs k, Fisher vs random
│       └── out/                      + CSVs and figures, each beside its script
└── tests/router/
    ├── test_skeleton.py              = import rules
    ├── test_data_*.py                + counts, NaN, price coverage, join rate, read-only Phase 5
    ├── test_sim_*.py                 + determinism, arrival events, metric values from §3.4
    ├── test_baselines.py             + each baseline on a toy matrix
    └── test_core_*.py                + IRT parameter recovery, anchor determinism, sd shrinkage

Still empty after M3: router/core/{memory,policy}.py (M4), router/live/ (M6),
router/proxy/ and router/dashboard/ (M7).
```

Two modules are new compared with the plan in Report 7: `data/prompts.py` and `data/scoring.py` come from findings R-a and R-b. `core/anchors.py` is split out of `irt.py`, because anchor selection is the comparison E1 now rests on (§5).

7.2 Main interfaces

```
# router/data/schema.py
@dataclass(frozen=True)
class Task:
    task_id: str
    dimension: str
    source: str               # e.g. "mbpp", "bcb", "lcb"
    prompt: str | None        # None when the join failed
    scoring: Literal["exec", "similarity", "judge"]

class OutcomeMatrix:
    tasks: dict[str, Task]
    models: list[str]
    score: pd.DataFrame       # task x model, NaN where a cell is missing (ragged view)
    cost: pd.DataFrame        # task x model, USD
    def split(self, name: str) -> "OutcomeMatrix": ...
    def without(self, model: str) -> "OutcomeMatrix": ...   # leave-one-model-out

# router/sim/stream.py
class Router(Protocol):
    def choose(self, task: Task, available: list[str]) -> str: ...
    def observe(self, task: Task, model: str, verified: bool, cost: float) -> None: ...

def replay(router: Router, m: OutcomeMatrix, *, seed: int,
           arrivals: dict[str, int] | None = None,
           verifier: Verifier) -> Trace: ...

# router/core/irt.py
def fit(scores: pd.DataFrame, model: Literal["1pl", "2pl"]) -> IRTParams: ...
def ability_posterior(params: IRTParams, outcomes: dict[str, int]) -> tuple[float, float]: ...

# router/core/anchors.py
def select(params: IRTParams, k: int, method: Literal["fisher", "random"],
           seed: int, theta_prior: float = 0.0) -> list[str]: ...

# router/core/onboarding.py
def onboard(model: str, anchors: list[str], probe: Callable[[str], int],
            params: IRTParams) -> tuple[float, float]: ...
```

`probe` is a callable so the same onboarding code serves both replay (look the outcome up in the matrix) and the live layer in M6 (call the model and run the tests).

7.3 Data flow through M3

1. `coderouterbench.py` and `prompts.py` build an `OutcomeMatrix` for each split and view.
2. `irt.py` fits task difficulties b and discriminations a on the probing split, with all eight models.
3. `difficulty.py` learns to predict b from the prompt embedding, so the router can score tasks it has never seen.
4. For a held-out model, `anchors.py` picks k probing tasks and `onboarding.py` turns the model's outcomes on them into $\hat{\theta} \pm$ sd.
5. `stream.py` replays the ID test split through each baseline, and `metrics.py` scores the trace. M4 adds our own router to this loop.

---

8. Planned Analyses and Expected Outputs

Each analysis below is a table or figure that M1 to M3 will fill. The cells are empty on purpose. The last column says what each outcome changes.

A1. Data integrity (M1)

Check	Expected	Observed	If it fails
Rows per split	56,640 / 23,352 / 1,408		Stop; the snapshot differs from the published one
Prompt join rate, per source family	≥ 95% overall		Exclude unjoined tasks from M3.2 and report the excluded share by dimension
Dimensions tagged by scoring type	9 of 9		R4 cannot be applied; report every result on all dimensions with a warning
Zero-token cost rows	148		Recheck the cost loader

A2. Baseline reproduction (M2 gate)

Router	Paper	Ours, ragged matrix	Ours, complete matrix	Δ (ragged)
Oracle	57.00	57.00 (§3.4)	60.02 (§3.4)	0.00
ACRouter (released decisions)	49.98 / 50.14		50.14 (§3.4)	
DimensionBest	47.50			
kNN retrieval	47.18			
LinUCB	46.84			
Random (10 seeds)	38.75			
Always-Opus	43.83		43.83 (§3.4)	

The figures already filled are the profile results from §3.4. The rest come from the M2 harness. Gate: Δ within ±1.0 for every row, or a documented definition difference.

A3. Cost at the best-single floor (M2, previews E2)

Policy	AvgPerf (95% CI)	$Total	CPAT@floor	Δ cost vs Always-Opus
Always-Opus				0
Fixed cascade Qwen3-Max → Opus, ε = 0				
Fixed cascade Qwen3-Max → Opus, ε = 0.2				
DimensionBest				
kNN				

The fixed cascade escalates on the simulated verifier, never on the raw score (R1). Our Phase 5 data showed that a fixed cascade beats the best single model on both pass rate and cost (Report 7 §2.2). This table tests whether that holds on a priced frontier pool. If no fixed cascade reaches the floor at lower cost, the sufficiency premise of the project is in trouble before M4. We would say so and report it.

A4. IRT model choice (M3)

Model	Response rule	Held-out log-likelihood per cell	Calibration error
1PL	binarised at 0.5		
2PL	binarised at 0.5		
1PL	binary dimensions only		
2PL	binary dimensions only		
Continuous variant	raw score		

The best row sets the IRT model and the acceptance rule for CPAT. If a single θ per model fits much worse than a θ per dimension, we switch to per-dimension abilities. The plan already lists that as the fallback.

A5. Difficulty from the prompt (M3)

Predictor	Spearman ρ (held-out b)	MAE
Dimension mean		
Source mean		
MiniLM embedding + ridge		
BGE embedding + ridge		

If the embedding predictors do no better than the source mean, Layer 2 will fall back to a source lookup plus the memory correction of M4. The pipeline stays the same either way.

A6. Ability error against k (M3, the early form of E1)

The main figure of this stage plots $|\hat{\theta}_k - \theta_{full}|$ against $k \in \{5, 10, 25, 50\}$. It has one line for Fisher anchors and one for random anchors, averaged over the eight held-out models and 20 seeds, with 95% bands. The matching table:

k	Fisher anchors: mean error (CI)	Random anchors: mean error (CI)	Fisher – random
5			
10			
25			
50			

This is the first test of contribution 1 (§5). If Fisher anchors do not beat random ones at small k, the anchor-selection claim is weak, and E1's weight shifts to the cascade and live results. We would report

that, not drop it (R2).

A7. Ranking check (M3 gate)

Held-out model	Full-data pass rate	$\hat{\theta}$ at k = 50	Rank by pass rate	Rank by $\hat{\theta}$
(8 rows)				

Gate: Spearman $\rho \geq 0.8$, with the mean absolute error of $\hat{\theta}$ reported beside it.

What Report 9 will contain that this report cannot

Report 9 will add the M4 policy, the onboarding regret curve (E1), the cost comparison against ACRouter (E2) and the verifier-noise study (E3). This report fixes their inputs: the matrix view, the scoring rule, the anchor method and the baseline numbers.

---

9. Threats to Validity Added This Period

- **Prompt reconstruction.** If the join fails for whole sources, the difficulty model is trained on a biased subset of dimensions. A1 reports the join rate per source so this stays visible.
- **Two matrices.** Results on the complete matrix are not comparable with the paper's tables, which use the ragged one. Every table says which matrix it uses.
- **Lost reference code.** Without the ACRouter scripts, a mismatch could be our bug or their undocumented choice. Scoring their released decision files separates the two, because it removes the router from the comparison.
- **Converted prices.** Five of the eight prices were converted from CNY. The R7 sensitivity sweep covers this.
- **Single-sample labels.** As before, each task-model pair has one recorded outcome, so oracle gaps include run-to-run noise.

---

10. Revised Timeline

Dates	Work	Status
5 Oct	M0 setup and novelty check	Done
6 Oct to 10 Nov	Planned M1 to M3	Not done
10 Nov	Data profile, requirements, design (this report)	Done
11 to 16 Nov	M1 data layer, prompt join, scoring tags	Planned
17 to 23 Nov	M2 replay, metrics, baselines, reproduction gate	Planned
24 Nov to 2 Dec	M3 IRT, difficulty, anchors, onboarding	Planned
December	M4 policy and E1 in the smallest form that answers the research question; thesis	Planned

The plan's priority order decides the cuts: E1, then E2, then the live web-development run E4, then E3, then the dashboard, then the proxy. With M3 now ending in early December, we expect:

- **Kept:** E1 on CodeRouterBench, and E2 in a reduced form (fewer seeds, no LLMRouterBench pool).
- **At risk:** E4, the live free-tier onboarding on the 65 web-development tasks.
- **Cut to fallback:** E3 to two noise levels; the dashboard to a static HTML report; the proxy to a command-line demo.

The submission and defence dates are not fixed yet. When they are, this timeline will be re-planned against them.

---

11. Next Steps

1. Download CodeRouterBench and LLMRouterBench into `external/`, and record the revisions.
2. Write the loaders and the 11 prompt-join adapters, then run A1.
3. Settle the scoring type of each dimension from the source benchmarks (FR4, C8).
4. Find out why API-013 and CSS-012 collected no tests on model code (§2.3).
5. Build the replay harness and baselines, then run A2 and A3.
6. Fit IRT, train the difficulty predictor, and run A4 to A7.

---

12. References

References [1] to [32] are as in Report 7, with these title corrections from the M0.5 check:

- [3] "RouterEval: A Comprehensive Benchmark for Routing LLMs to Explore Model-level Scaling Up in LLMs," arXiv:2503.10657.
- [14] "Learning to Route LLMs from Bandit Feedback: One Policy, Many Trade-offs," arXiv:2510.07429.
- [15] "Learning to Route and Schedule LLMs from User Retrials via Contextual Queueing Bandits," arXiv:2602.02061.
- [16] "The Routing Plateau: Understanding the Accuracy Limits of LLM Routers," arXiv:2606.07587.
- [17] "Most of the LLM Routing Gap Is Task Type," arXiv:2608.23023.
- [19] "RouteProfile: Graph-Based Profiling for Cold-Start LLM Routing," arXiv:2605.00180.
- [25] "Beyond GPT-5: Making LLMs Cheaper and Better via Performance-Efficiency Optimized Routing" (Avengers-Pro), arXiv:2508.12631.

All identifiers previously marked *(unverified)*, [26] and [28] to [31], resolve to the cited papers. The arXiv metadata confirms the venues of [28] (NeurIPS 2024), [29] (NAACL 2025) and [31] (NeurIPS 2024). The ICLR 2025 venue of [26] is not in the arXiv metadata and remains to be checked.

New references from the novelty check:

[33] "ICL-Router: In-Context Learned Model Representations for LLM Routing," arXiv:2510.09719, 2025. [34] "FlyRoute: Self-Evolving Agent Profiling via Data Flywheel for Adaptive Task Routing," arXiv:2605.22057, 2026. [35] "Routing with Generated Data: Annotation-Free LLM Skill Estimation and Expert Selection" (CASCAL), arXiv:2601.09692, 2026. [36] "ORACLE: Agentic AI Orchestrator Routing Via Adaptive Verifier Calibration Feedback," arXiv:2607.22465, 2026.

Data: CodeRouterBench, Hugging Face dataset `Lance1573/CodeRouterBench`, revision of 2026-06-23, MIT licence, accessed 10 November 2026.

Titles [33] to [36] were checked against the arXiv API on 10 November 2026.